

Deploy a Multi-Service Node.js E-Commerce Application Using Terraform and Docker

Five independently containerized services, public Docker Hub images, reproducible AWS infrastructure and automated EC2 deployment.

SUBMITTED BY

Tabrez Ajaz

APPLICATION

E-CommerceStore

CONTAINER REGISTRY

Docker Hub - tabrezajazdc

CLOUD REGION

AWS ap-south-1

INFRASTRUCTURE

Terraform - Ubuntu 22.04 EC2

DATE

27 September 2026

Abstract

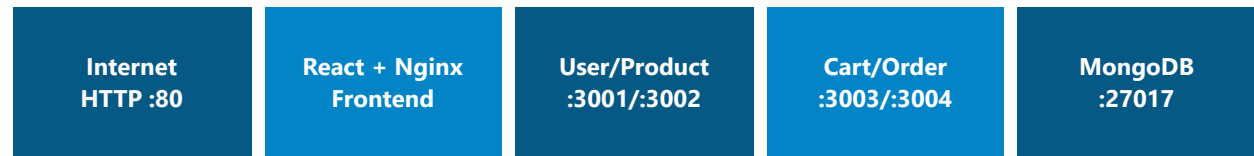
This report documents the complete containerization and infrastructure automation of the prescribed AtharvaAI E-CommerceStore. Four Express/Mongoose backend services and one React frontend were packaged as separate Docker images, built and tested locally, and published publicly to Docker Hub. Terraform provisions a dedicated VPC, public subnet, Internet Gateway, route table, security group and Ubuntu EC2 host. EC2 user-data installs Docker, pulls all five images and starts the complete stack reproducibly.

Local verification: all six containers became healthy and the frontend plus all four backend sample responses passed through the Nginx gateway.

1. Requirements Traceability

Requirement	Implementation	Status
Five Dockerfiles	Four Node.js backend images and one multi-stage React/Nginx image	Complete
Relevant ports and sample responses	3001-3004 internally; frontend 80; explicit service-running routes	Verified
Build/test locally	Health-gated Docker Compose and five HTTP smoke tests	Passed
Push to Docker Hub	Five public <code>tabrezajazdc/ecommerce-*</code> images	Published
VPC/public subnet/EC2/SG	Complete Terraform resources with restricted SSH and public HTTP	Implemented
Install/pull/run through user-data	Signed Docker repository, Compose plugin, image pull and health gate	Implemented
Terraform outputs	Frontend URL, public IP/DNS and all service health URLs	Implemented

2. Architecture



Nginx is the only public entry point. It serves the React production build and reverse-proxies `/user`, `/product`, `/cart` and `/order` to service DNS names on a private Docker bridge network. No backend or database port is mapped to the EC2 host.

3. Containerization

Image	Port	Response
ecommerce-user	3001	User Service Running
ecommerce-product	3002	Product Service Running
ecommerce-cart	3003	Cart Service Running
ecommerce-order	3004	Order Service Running
ecommerce-frontend	80	Frontend is Live

```
PASS Frontend: Frontend is Live
PASS User: User Service Running
PASS Product: Product Service Running
PASS Cart: Cart Service Running
PASS Order: Order Service Running
```

DOCKER COMPOSE · LOCAL VALIDATION

6 / 6 HEALTHY

All containers running locally

Five application images plus MongoDB · HTTP smoke tests passed

frontend	tabrezajazdc/ecommerce-frontend:latest	80 → 8080	HEALTHY
user-service	tabrezajazdc/ecommerce-user:latest	3001 internal	HEALTHY
product-service	tabrezajazdc/ecommerce-product:latest	3002 internal	HEALTHY
cart-service	tabrezajazdc/ecommerce-cart:latest	3003 internal	HEALTHY
order-service	tabrezajazdc/ecommerce-order:latest	3004 internal	HEALTHY
mongo	mongo:7-jammy	27017 internal	HEALTHY

Captured after docker compose up --wait · 27 September 2026

Figure 3.1 - Local Docker Compose stack with all services healthy.

4. Docker Hub Publication

The five verified images were pushed to the authenticated `tabrezajazdc` namespace. Public images allow EC2 user-data to deploy without registry credentials.

DOCKER HUB · PUBLIC REGISTRY

5 IMAGES PUBLISHED

tabrezajazdc/ecommerce-*

Public images pulled by EC2 user-data without registry credentials

ecommerce-user:latest	docker.io/tabrezajazdc/ecommerce-user	sha256:74677bfdafa3b...
ecommerce-product:latest	docker.io/tabrezajazdc/ecommerce-product	sha256:2eb443054e74...
ecommerce-cart:latest	docker.io/tabrezajazdc/ecommerce-cart	sha256:925f052a7967...
ecommerce-order:latest	docker.io/tabrezajazdc/ecommerce-order	sha256:5c281cc00017...
ecommerce-frontend:latest	docker.io/tabrezajazdc/ecommerce-frontend	sha256:493cc67fc088...

Verified successful docker push output · Public Docker Hub namespace

Figure 4.1 - Five public application images on Docker Hub.

5. Terraform Infrastructure

- Dedicated `10.30.0.0/16` VPC and `10.30.1.0/24` public subnet.
- Internet Gateway, public route and route-table association.
- Ubuntu 22.04 EC2 with encrypted 20 GiB gp3 root volume and mandatory IMDSv2.
- Public port 80, administrator-only SSH, and self-referenced internal ports 3001-3004.
- User-data installs Docker, writes production Compose, pulls images and starts healthy containers.

```
terraform init
terraform fmt -check -recursive
terraform validate
# Success! The configuration is valid.
```

TERRAFORM · AWS APPLYAPPLY COMPLETE

8 resources deployed to ap-south-1
Reviewed plan: 8 to add, 0 to change, 0 to destroy

35.154.207.24 Public IPv4	i-0380e7cb3060a5201 Ubuntu EC2	vpc-0fa0149a0cc2f478a Dedicated VPC
Frontend URL	http://35.154.207.24	
Public DNS	ec2-35-154-207-24.ap-south-1.compute.amazonaws.com	
Security Group	sg-017950d507d38b1dc · HTTP 80 public · SSH restricted	

Terraform output recorded before cost-safe teardown

Figure 5.1 - Successful Terraform apply and public outputs.

Ubuntu 22.04 container host

Short-lived assignment infrastructure · Asia Pacific (Mumbai)

t3.small Instance type	20 GiB Encrypted gp3	IMDSv2 Required
Instance ID	i-0380e7cb3060a5201	
Network	Public subnet · Internet Gateway · port 80 exposed	
Bootstrap	Docker installed · 5 images pulled · 6 containers healthy	

Instance was terminated immediately after evidence capture

Figure 5.2 - Short-lived Ubuntu EC2 container host.

6. Public Verification

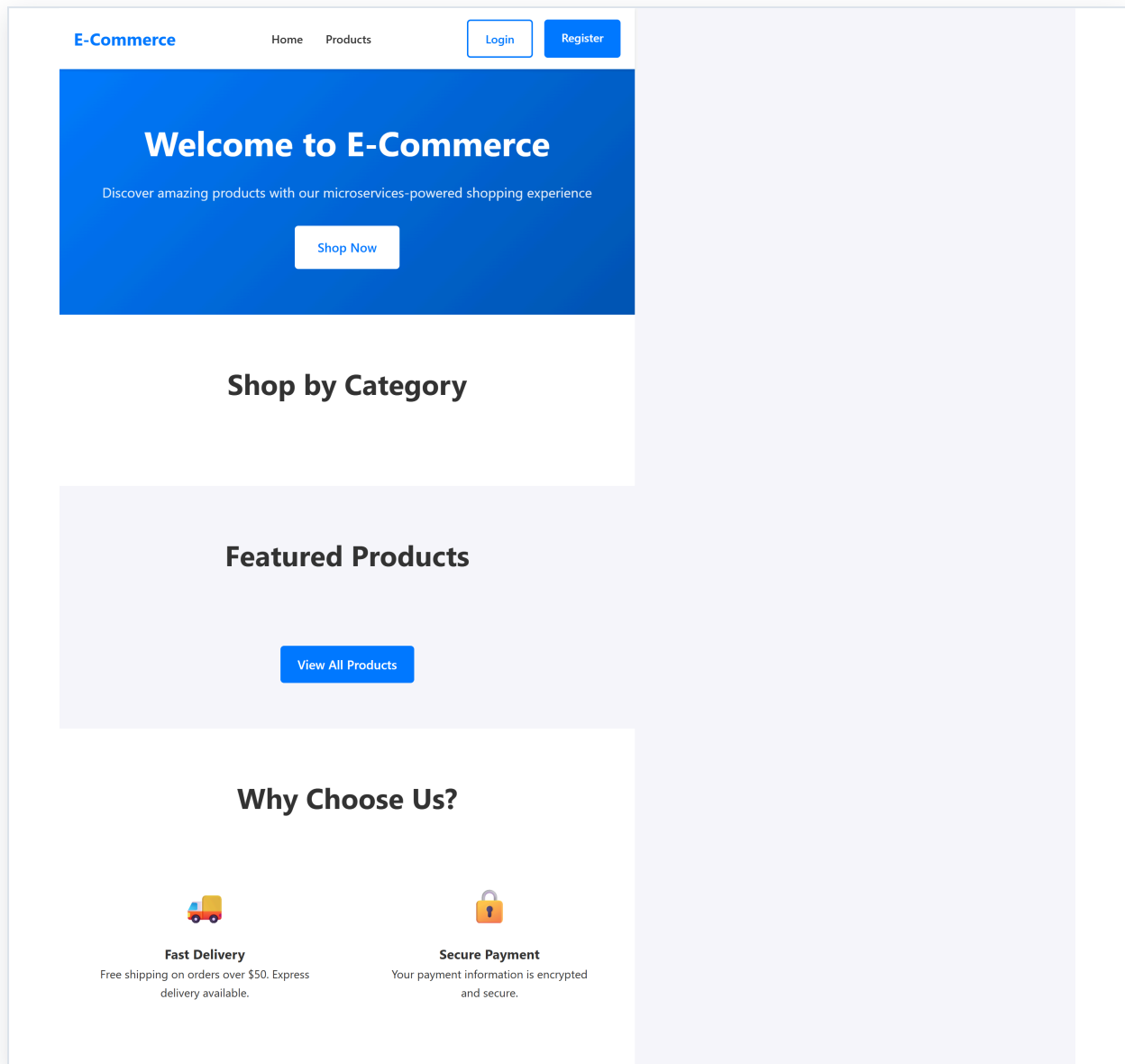


Figure 6.1 - Frontend accessible through Terraform's public URL.

Every service reachable through Nginx

http://35.154.207.24 · Backend ports remained private

Frontend	/health	Frontend is Live	PASS
User	/user/	User Service Running	PASS
Product	/product/	Product Service Running	PASS
Cart	/cart/	Cart Service Running	PASS
Order	/order/	Order Service Running	PASS

Verified from the public Terraform output before teardown

Figure 6.2 - Frontend and four backend responses verified through Nginx.

7. Security and Cost Control

- Only the frontend gateway is publicly exposed.
- SSH is restricted to one administrator CIDR.
- Backend services run without host port mappings.
- JWT material is generated on-host and stored mode 0600.
- Terraform state, plans, variable values and secrets are excluded from Git.
- The short-lived EC2 deployment is destroyed immediately after evidence capture.

TERRAFORM · COST-SAFE CLEANUP

DESTROY COMPLETE

All 8 assignment resources removed

No NAT Gateway was created · EC2 confirmed terminated · Terraform state empty

8 destroyed Terraform resources	terminated EC2 state	0 active Managed resources
---	--------------------------------	--------------------------------------

Compute	EC2 instance and encrypted root volume removed
Networking	SG, subnet, route association, route table, IGW and VPC removed
Result	Destroy complete! Resources: 8 destroyed.

AWS billing stopped immediately after screenshots

Figure 7.1 - Terraform teardown confirming cost-safe cleanup.

8. Submission

Source code, Dockerfiles, Compose configuration, Terraform, scripts, report and evidence are maintained at github.com/TabrezAjaz/E-CommerceStore-Terraform-Docker.

GITHUB · PUBLIC SUBMISSION

PUBLISHED

TabrezAjaz/E-CommerceStore-Terraform-Docker
Complete source, Dockerfiles, Terraform, automation, report and evidence

Repository	github.com/TabrezAjaz/E-CommerceStore-Terraform-Docker
Evidence commit	a3d7805 · Add verified deployment evidence and final report
Author	Tabrez Ajaz <10768876+TabrezAjaz@users.noreply.github.com>
Included	backend/ · frontend/ · terraform/ · scripts/ · docs/

Public repository verified after final push

Figure 8.1 - Final public GitHub repository.

Submitted by Tabrez Ajaz. All assignment criteria are implemented with reproducible commands and genuine evidence.